

Old Wine in New Bottles: Using Classical Word Embeddings in Gate-Based Quantum NLP Systems

Damir Cavar(✉), Koushik Reddy Parukola, James Bryan Graves, and Shane A. Sparks

Indiana University, Bloomington, IN 47405, USA
dcavar@iu.edu

Abstract. Vector representations as embeddings of words, text, or images in n-dimensional semantic space are essential for neural models in state-of-the-art (SOTA) Natural Language Processing (NLP) and AI systems. Such embedding models are engineered and trained using classical computers and large amounts of data. Given the current technical limitations, we are unaware of a method to achieve the same task on a quantum computer alone. In this report, we discuss experimental results related to hybrid systems utilizing classical models in quantum algorithms for common NLP and AI inferencing. We demonstrate how word and text embeddings as n-dimensional dense vectors of real numbers, as used in current classical SOTA AI systems, can be mapped to quantum states using $log_2(n)$ and $log_2(n/2)$ qubits. The quantum encoding strategies used for this task include amplitude encoding of real and complex word embeddings to quantum state vectors, as well as reductions of Hamiltonian representations of quantum states to further reduce the number of required qubits, the number of gates, and the circuit depth. We show that the mapping results in a linear number of circuit gates. The high correlation coefficient between the classical and quantum embedding similarity scores indicates that there is no significant information loss that would affect the semantic space distribution of words in the tested quantum encoding strategies. The results show that existing NLP embedding models in classical computing environments can be used efficiently in quantum computing. We discuss how improvements of classical embedding models could significantly improve the classical to quantum computing interface for popular systems, such as Generative AI and Large Language Models (LLM).

Keywords: Quantum Computing · Natural Language Processing · Word Embeddings · Computational Semantics

Supported by the Center for Quantum Technologies and the Indiana University Pervasive Technology Institute.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Cavar et al. (Eds.): QNLPAI 2025, CCIS 2659, pp. 156–171, 2026.
https://doi.org/10.1007/978-3-032-13883-5_9

1 Introduction

Generative AI and Large Language Models (LLMs) rely on vector representations of linguistic units, for example, subword tokens or entire words. Such vectors or embeddings are crucial for all SOTA NLP and AI algorithms based on a neural architecture or transformer models. From a technical perspective, using fixed-length vector representations of words or text eliminates the issue of handling variable-length input. It also reduces the problem of handling words and text that have not been included in the training data of the NLP and AI models.

Neural network architectures and trained models have a predefined architecture for input and output that cannot be altered in the training phase of the models or in the final application and induction phases. Currently, all SOTA neural models require input vectors with a specific dimensionality. Since natural language expressions vary in length, encoding them as fixed-length vectors simplifies their use in neural models, in particular, in current SOTA LLMs.

Vectors that act as embedded approximations of word meanings in a semantic vector space are typically trained using distributional semantic assumptions [17]. The distributional properties of words extracted from text corpora provide the empirical basis for training dense vector representations in a self-supervised setting, capturing aspects of word meanings such as similarity in common contexts.

The idea of using contextual and distributional properties of words can be traced back to several publications. According to [13], the distributional properties of the linguistic units can be derived from their contextual usage. [11] suggested that the context of an ambiguous word can be used for word sense disambiguation. [30] proposed performing word sense disambiguation leveraging the distributional properties of ambiguous words in large corpora.

Using vector spaces to model the semantic properties of lexical items is much older than current technical implementations. [27], for example, suggested a model based on multidimensional Euclidean space. Analogies in such a model would correlate with the relative distances of the terms measured as the vector distances.

Current SOTA word embedding models thus provide vector representations for linguistic units that are geometrically similar to other word embeddings if they share similar contexts in text corpora used for training. For example, *cat* and *dog* share more contexts than *cat* and *moon*, and thus the vector representations for *cat* and *dog* should be closer in Euclidean space than *cat* and *moon*. In the early phases of vector space modeling of linguistic units, these vectors were, for example, handcrafted and encoded specific features that are geared towards optimization in specific NLP or AI applications, e.g., text classification, sentiment analysis, or grammar checking.

To train such word embeddings, the frequencies of terms and co-occurrence in fixed-size context windows in documents are used. Some models are trained to predict context words from specific target words [3,20,23], while others are trained to predict a masked word given a specific context [9]. In LLMs, embeddings

are trained to predict the next word given a previous context of n words [4]. The distributional properties and frequencies of contextual collocations of words are mapped into probability distributions to train the word embeddings. These frequency-based vectorization approaches found their application in the lexical-semantic modeling of words and lexical units in NLP applications. Instead of using a specific vector length of arbitrary real values or some weighting model over frequency and collocation distributions of words, such vectors are trained using Distributional Semantics [17], as described above.

That is, the vectorized representations of words in the form of dense vectors are optimized to reflect semantic properties that can be measured in terms of similarity by using term or word collocations in close proximity. In such models, semantically related words are closer to each other than semantically unrelated words, which are measured using linear algebra concepts. The similarity in these linguistic units can be measured using, for example, *cosine similarity* or *inner product* between dense vectors (i.e., word embeddings).

Figure 1 shows the similarity and analogy relations between word pairs in the GloVe [23] embedding space:

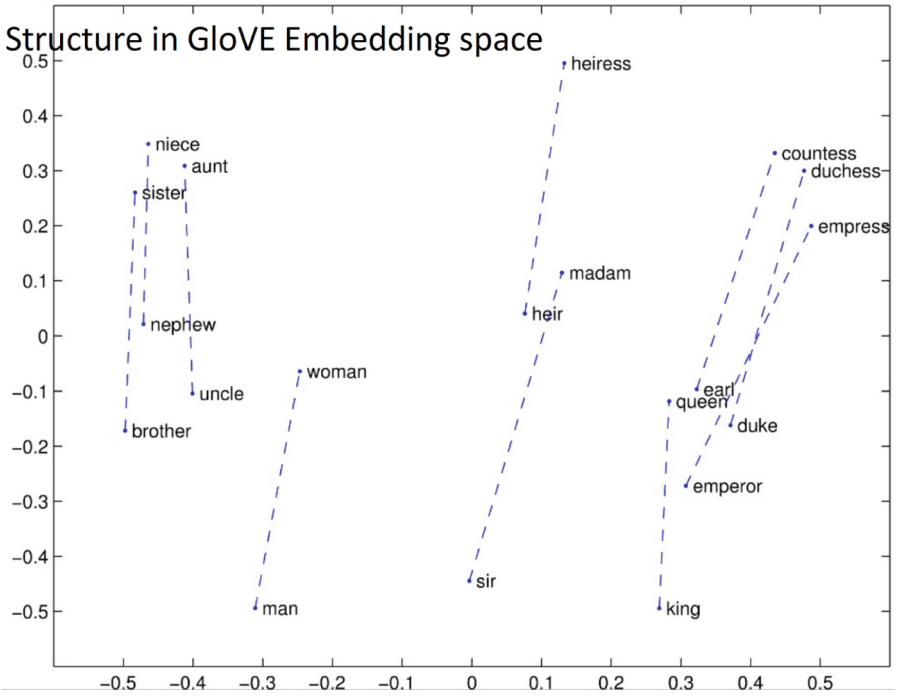


Fig. 1. Similarity and Analogy in the GloVe Embedding Space [23]

One of the goals in the GloVe model was to add distributions reflecting analogy to the semantic space as discussed in [28]. Analogy in this approach

could be understood as the spacial distance of two parallels, e.g., "brother is to sister what nephew is to X" with X being identified as the approximately equidistant concept on the parallel to the "brother - sister" vector.

Such embedding models are limited to n-dimensional vectors in classical computing environments. In Quantum Natural Language Processing (QNLP) or Quantum Machine Learning (QML) [25], such models could be based on much more sophisticated mathematical grounds. In particular, we could think of predicates from a theoretical semantic perspective as different from referential nominal entities or objects. In our models, we treat predicates as quantum gates, i.e., as matrices rather than vectors (see, for example, [2]).

In QNLP, a potential quantum advantage might be achieved for semantic vector space models using the extended modeling possibilities that quantum computing environments provide [8].

Classical NLP approaches use n-dimensional vectors as word embeddings. Such embeddings could be used in a quantum computer to perform common NLP tasks, such as text classification. However, quantum vector spaces, being physical Hilbert spaces of quantum states [10], provide a much broader framework for modeling semantic properties of natural language units. In NLP, vector space modeling for semantic properties [16] does not use this broad capacity of quantum computing environments.

While our research goal aims at these extended modeling opportunities for quantum embeddings and linguistic semantics, we focus our work here on experiments related to the interface between classical and quantum computers and how machine learning models from classical computers could be transferred into the quantum environment by minimizing loss of information and maximizing usability for NLP and AI applications.

Given the effort and costs associated with generating language models and word embeddings, reusing existing models is naturally a priority if it is technically feasible and if no significant information loss in the embedding model is associated with the transfer from classical to quantum computers. The work here focuses on exactly that, i.e., evaluating the semantic loss involved in encoding classical word embeddings as quantum states.

2 Previous Work

[26] demonstrate that encoding methods that map dense classical vectors to quantum embeddings, e.g., basis encoding, angle encoding, or amplitude encoding used in popular machine learning algorithms, can not only be used successfully in quantum computing, but that the resulting quantum embeddings can in fact "contribute to improved classification accuracy and F1 scores" in the relevant applications.

[18] use amplitude encoding for words in a binary sentence classification task, deriving the word meaning using two different quantum-inspired models. They utilize the Hilbert Space representation of particles in Quantum Mechanics, assigning a relative phase (a complex number) to each word.

A recent paper discusses quantum embeddings in the context of computer vision [34]. While this is similar to our work here only with respect to using quantum embeddings for semantic representations, it is not using quantum embeddings for words or text representations.

Although encoding methods such as amplitude encoding or angle encoding have been reported to be adequate for QML applications [26], to the best of our knowledge, there are no experimental results measuring information loss related to semantic similarity in the different encoding methods used to transfer machine learning models such as classical embedding vectors to a quantum computer that encodes word embeddings as quantum states.

Previous experiments of quantum embeddings from classical embeddings have shown that there is a strong correlation between the similarity scores for classical word pair embeddings and their quantum embedding counterparts using amplitude encoding, e.g., [6,7]. The similarity scores in this study were computed for a few arbitrarily selected nouns.

3 Quantum Word Similarities: Methodology

Classical word embeddings are n-dimensional vectors of real numbers. To compare the similarities of two words in the classical environment within one model, we use linear algebra computations based on the embedding vectors for those words. Transferring classical vectors to quantum environments requires an encoding strategy, such as basis encoding, angle encoding, or amplitude encoding [21,26] used in popular machine learning algorithms [19].

For the comparison between vector similarities in classical and quantum environments, we use classical word embedding models. We extract vector representations for individual words using all these models. Our particular focus is on GPT embeddings offered by OpenAI, and VoyageAI embeddings as used in Claude 3.x AI models by Anthropic. The vectors from these models are used to compute the similarity score on classical computers. Subsequently, we encode these vectors as quantum embeddings using amplitude encoding alongside two different vector reduction approaches to reduce the vector dimensionality to 50% of its original size. We then measure the quantum similarity between two quantum embeddings using the SWAP test [1,5].

The classical similarity between two words was computed in our experiments using cosine similarity and, if not normalized, also the model-specific embedding vectors for those words; see Eq. 1.

$$cos(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}||\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2} \sqrt{\sqrt{\sum_{i=1}^{N} w_i^2}}} \tag{1}$$

Some models, e.g., the *voyage-3* embeddings by VoyageAI (as recommended by Anthropic, Claude-3.x), are normalized. Thus, the dot product similarity

score with *voyage-3* embeddings was computed; see Eq. 2. In the classical computing environment, the vectors were not manipulated, normalized, or padded.

$$\vec{v} \cdot \vec{w} = \sum_{i=1}^{N} v_i w_i \tag{2}$$

For quantum embedding of the classical word vectors, the vectors were normalized and padded to a length that is a power of 2. All word vector models used have been preprocessed in this way for the quantum similarity experiments, as to reduce the computational effort in the quantum embedding computations.

Given the normalized and padded word embedding vectors, the quantum encoding was performed using amplitude encoding. With a normalized and 0-padded vector v of real numbers and 4 dimensions, each scalar is used to specify one amplitude of a qubit superposition.

$$\vec{v} = \begin{pmatrix} 0.27 & 0.54 & 0.81 & 0.0 \end{pmatrix} \tag{3}$$

With one qubit represented as in Eq. 4, we have two basis states 0 and 1, each associated with one complex amplitude α and β respectively. For n qubits there are exactly 2^n basis states and one corresponding amplitude for each basis state.

$$|\Psi\rangle = \alpha|0\rangle + \beta|1\rangle \tag{4}$$

With each additional qubit, the number of amplitudes grows exponentially. With two qubits, as in Eq. 5, there are four amplitudes.

$$|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle \tag{5}$$

The amplitude encoding of normalized real vectors replaces each amplitude with a dimension from the vector in sequence, as in Eq. 6 for vector v.

$$|\psi\rangle = 0.27|00\rangle + 0.54|01\rangle + 0.81|10\rangle + 0.0|11\rangle \tag{6}$$

While the binary encoding of such vectors in classical computers usually requires $n \times 32$ *bits*, i.e., 4×32 *bits* $= 128$ *bits* in our example above, such a 128 *bit* classical representation can be encoded and significantly compressed in just two or even only one qubit, as explained in the following.

As mentioned above, the amplitudes of qubits are complex numbers. In Eq. 6, we ignore the imaginary part by setting it to $0i$. This method achieves a compression of n-dimensional real vectors to $log_2(n)$ qubits. Encoding the dimensions of a real vector as complex amplitudes can be achieved by setting every second dimension as the imaginary part of one amplitude. Thus, our vector in Eq. 3 could be encoded in just one qubit since we only need $log_2(n/2)$ qubits, as shown in Eq. 7.

$$|\psi\rangle = (0.27 + 0.54i)|0\rangle + (0.81 + 0.0i)|1\rangle \tag{7}$$

Using this encoding strategy and word embeddings from OpenAI's model *text-embedding-ada-002* with 1536 dimensions, the real-number amplitude encoding would require 11 qubits (since $\lceil \log_2(1536) \rceil = 11$), while the complex-number amplitude encoding would require 10 qubits (since $\lceil \log_2(1536/2) \rceil = 10$).

Saving only one qubit in the complex-amplitude encoding approaches does not seem to be significant. However, with an input vector length of N, the circuit depth and the number of gates are $O(N)$. For each dimension of an embedding vector, we might need a time step in a quantum circuit, which corresponds to the circuit depth. Along the same line, for each amplitude, we need a gate to set it in the circuit. This means that the number of gates and the circuit depth both scale linearly with the length of the embedding vector. The complex-number amplitude encoding reduces the number of amplitudes that need to be set by half. This significantly reduces the number of gates and the circuit depth required for amplitude encoding. Given that current qubit coherence times are challenging, and the probability of errors grows with the accumulation of errors as the circuit evolves, the complex-number amplitude encoding would be a more desirable encoding approach.

The experiments described in the following aim to clarify whether these quantum encoding methods are problematic in maintaining the semantic properties of the word embeddings from the different AI models.

Our approach to defining the quantum similarity between two words is based on the SWAP test [1,5]. The SWAP test can be performed between any two circuits S and T of the same number of qubits and can measure the difference between S and T. It requires one additional ancilla qubit and a classical bit for the measure. This test measures the similarity between two quantum states $|\psi\rangle$ and $|\phi\rangle$ by estimating the squared inner product of the two quantum states. The central object of the SWAP test is the SWAP operator, a time-evolution operator under the Heisenberg exchange Hamiltonian $H = X \otimes X + Y \otimes Y + Z \otimes Z$, for a time $t = \pi/4$ (up to a global phase), or $SWAP = e^{-iH\frac{\pi}{4}}$. This SWAP can be controlled, $H_{CSWAP} = |1\rangle\langle 1| \otimes H$. Thus quantum dynamics are leveraged in our experiment by means of the SWAP tests, which are fundamentally interferometric measurement of overlap, implemented via controlled exchange dynamics.

Applying the SWAP test in our experiment means that we generate two quantum states for the two word vectors `word1` and `word2`. We denote the corresponding quantum circuits `circuit(word1)` and `circuit(word2)`, we define the SWAP distance `swap_dist(word1, word2)` between `word1` and `word2` to be

$$\begin{aligned} &\texttt{swap_dist(word1, word2)} \\ &= P(1; \texttt{circuit(word1), circuit(word2)} \end{aligned} \tag{8}$$

which is the probability of measuring 1 from q_0 with $S = \texttt{circ(word1)}$ and $T = \texttt{circ(word2)}$

In the following, we describe in more detail the selection of words from the different classical word embedding models used in the evaluation.

4 Data and Experiments

The data set that we used in the experiments consists of nouns from the SimLex-999 dataset [14]. SimLex-999 contains 999 word pairs with human-assigned similarity scores, with two-thirds of these being noun pairs. With this, there are

666 noun pairs in use for this experiment. We computed the similarity scores for these noun pairs using a variety of embedding models and quantum embeddings.

We restricted the selection of words and the corresponding embeddings from the different models to only nouns, of which there are a unique 751 within SimLex-999, as some nouns are present multiple times in different pairs. The reason for restricting the selection is that we assume predicates to be more accurately modeled not as vectors but rather as functions, operators, or, in our context, as quantum gates.

Measuring a vector embedding of nouns is more reasonable from the perspective of a classical set-theoretic semantic theory, even though the embedding models are trained using a simplified distributed semantics assumption, which does not differentiate between predicates with an argument structure and contextual arguments. Because of this method of training, the experiment and its results, we argue, are independent of the choice of words and our restriction to nouns only.

The approach in this experiment is to pick words from a model and compute similarity scores for word pairs using the embeddings of these words. After encoding the embeddings into quantum states, we measure the similarity scores of the quantum states using the SWAP test. We analyze the correlation coefficient between the classical similarity scores of word pairs and the quantum similarity scores. A high correlation coefficient would indicate little information loss.

We picked the 666 noun pairs from SimLex-999 since we could use the individual word embedding vectors from the different models. Some embedding models are static, i.e., there will be no embedding for out-of-vocabulary (OOV) words. Other models can compose an embedding for OOV words from tokens that are common subword tokens of words (see, for example, [9]). Current SOTA LLMs use byte-pair encoding (BPE) that is not based on common linguistic intuitions about morphemes, subword elements, or complete words. Such BPE-based models can generate embeddings as a composition of multiple subword tokens based on BPE tokenization.

For the experiments, we used all 666 noun pairs from SimLex-999, instead of using the complete set of 576,081 possible noun pairs from the Cartesian product. We decided to pick this existing subset with human similarity scores to reduce the huge computational effort associated with similarity computation, in particular when using quantum simulators or real quantum computers.

We computed the cosine similarity and the quantum state similarity for all pairs using the vector representations from the models listed below, as well as using the complex amplitude encoding. The quantum state in this experiment represents the quantum embeddings for two nouns using the amplitude encoding of a classical embedding vector.

We used the following models in previous studies to select word pairs and measure classical similarities (through cosine similarity) using the model embeddings. All models except ConceptNet Numberbatch [31] were English word lists, but many of those contained non-English words and non-words or non-linguistic

expressions. The Numberbatch model, trained on the ConceptNet knowledge graph, is multilingual. BERT is not a model consisting of word and vector pairs, but rather a transformer [33], which is a deep learning [12] model that generates a vector representation for some input text. We used BERT word embeddings without providing any context words.

- word2vec, 300-dim. word vectors, [20]
- GloVe, 840 billion tokens, 300-dim. word vectors, 2.1 mil. words, [24]
- fastText, 300-dim. word vectors, 2.5 mil. words, [3,15]
- Numberbatch, 300-dim. vectors, 516,783 words, [31]
- BERT, 768-dim. word vectors, [9]
- GPT embeddings, 3072-dim. *text-embedding-3-large* and 1536-dim. *text-embedding-3-small* model, see https://platform.openai.com/docs/guides/embeddings
- Anthropic/VoyageAI embeddings, model *voyage-3*, 1,024-dim.,[1] see https://docs.voyageai.com/docs/embeddings

We expect differences in these models with respect to word similarities, as reflected in the geometric distance of the embedding vectors. Meanwhile, word2vec vectors were generated using continuous bag-of-words (CBOW) and continuous skip-gram to train the word embeddings. In the CBOW algorithm, a specific word is predicted given the context, while in the skip-gram algorithm, the words in a local surrounding context are predicted. The resulting word embeddings are not disambiguated, in that the word *bank* just has one vector representation which carries all potential meanings for the word, with these different meanings occurring in differing contexts. For example, the meaning *financial institution* might occur in a context mentioning *money*, *account*, or *customer*, while the meaning *river bank* would more frequently be surrounded by words like *river* and other terms of nature. The GloVe model improves this approach by learning word embeddings from global co-occurrence statistics of words in a large corpus. It improves word analogy learning compared to word2vec, as shown in Fig. 1.

The fastText model further improves the embeddings by adding subword embeddings. Subword embeddings provide vector representations for morphemes that are segments in words. For example, the word *unlockable* would have the sequence of *un*, *lock*, and *able* as subwords. This approach seems to improve the embedding properties for rare words or out-of-vocabulary words for which an embedding can be computed from the word segments.

The Numberbatch word embeddings are built using ConceptNet, which is a knowledge graph representation of common sense concepts and terms, and embeddings from word2vec and GloVe. Since ConceptNet provides relations between concepts (or words that represent concepts) such as hypernym, synonym, or related edges, this information significantly improves the semantic properties of the Numberbatch embeddings, outperforming word2vec and GloVe

[1] *voyage-3* vectors are normalized to length 1, thus dot product for classical similarity was used.

with respect to word similarity metrics measured in terms of geometrical vector distances.

BERT is trained using the masked word approach. This approach is comparable to a cloze test [32], where a human language learner must guess a masked or missing word in a specific context. One of the core advantages of the BERT model is that word embeddings for specific words are dynamically computed depending on the context, providing *contextually informed word embeddings*. Since we are not using context in our experiments here, we always generate the same vector representation for a word using BERT. In subsequent experiments, we intend to test multiword expressions and more complex utterances, which would benefit from this specific property of BERT.

Another crucial difference between BERT and other discussed embedding models is that BERT uses the subword tokenization algorithm WordPiece to generate embeddings for subwords. A word like *locking* could be tokenized as *lock* and *##ing*, and embeddings would be provided for both word segments (or tokens). The special markup of the token *##ing* indicates that *ing* should be treated as a word continuation in the sense of a dependent morpheme in the linguistic sense. This way, BERT reduces the unknown word problem and increases the coverage at the lexical level. The WordPiece algorithm was first laid out in [29] as a method to cope with out-of-vocabulary words and improve the general performance of machine learning and NLP tasks.

Given the fundamentally different underlying machine learning approaches and significant differences in the amount of data and text corpora used to train the different models, we also expect a very different overall similarity score in the classical and quantum embeddings.

None of the classical vectors was reduced or, in other ways, manipulated. We did not fine-tune any of the models and restricted ourselves to the raw models as pre-trained by the providers. The goal here is not to tune or improve the embeddings for specific tasks but rather to evaluate whether their properties can be transferred to a quantum computing environment with minimal loss.

5 Evaluation of Similarities

The evaluation steps in this experiment involve:

- Generating cosine similarity (or dot product) scores for all word pairs using all the different models.
- Using amplitude encoding to convert word embedding vectors from the classical computing environment to quantum states.
  - Using real-number amplitude encoding
  - Using complex-number amplitude encoding with two different real-to-complex conversion strategies
- Generating the quantum state similarity scores using SWAP-test-based quantum circuit/state similarities.
- Computing the correlation coefficient between the classical similarity scores and the quantum similarity scores.

There were multiple variables that we could control. Due to the complexity of the experiment and the computing time for the interface between classical and quantum computing, we ran only one variable setting. The quantum measures were based on the similarity calculation of 1,000 shots per word pair. We only used the `qasm_simulator` from `qiskit-aer 0.16.1` in the initial phase of the experiment. Currently, we are preparing to run the same experiments on the IBM Quantum computers.

As mentioned earlier, given the amplitude encoding approach, we can compress a 300-dimensional word embedding vector with real number values to eight or nine qubits, depending on the strategy. This means that, on average, a $300 * 32\ bit = 9,600\ bit$ vector can be compressed to just nine qubits. The corresponding 768-dimensional BERT vectors can be represented as ten-qubit embeddings. Even a 3,072-dimensional GPT *text-embedding-3-large* embedding can be converted to an 11- or 12-qubit quantum state. This compression rate is highly significant. Existing quantum computers allow us to experiment on real hardware with these embeddings, given that the SWAP test requires a maximum of 25 qubits. Our goal is to evaluate whether there is some information loss as measured by similarity after mapping classical vectors to quantum states.[2]

6 Results

When interpreting the correlation results between the classical similarity scores with one embedding model and its quantum counterpart, we should be aware of some correlation coefficients between the classical models. For example, if we compare the correlation coefficients for the same word pair similarity scores between fastText and Glove vectors, we achieve a correlation coefficient of 0.84.

For the selected 666 noun pairs and quantum-to-classical similarity correlations, we focus on SOTA embeddings from OpenAI for GPT and from VoyageAI.

The first correlation coefficients that we compare are for OpenAI's GPT embeddings. Table 1 shows the correlation coefficients for the different similarity scores.

Table 1. GPT embeddings correlation coefficients

	Classic	Q Real	Q Comp.int.	Q Comp.split
Classic		0.9485	0.9076	0.9099
Q Real			0.8914	0.8821
Q Comp.int.				0.8590

Classical similarities (*Classic*) using cosine similarity for the GPT embeddings correlate closest to the quantum embeddings generated using amplitude

[2] See the Limitations section for some problems with quantum embeddings based on amplitude encoding.

encoding with real numbers. The correlations are also very high between the classical embeddings and the quantum complex embeddings. The given correlation coefficient above 0.9 means that the semantic similarities between the words in the word pairs are mostly preserved. The quantum real-number encoded states show a correlation coefficient less than 0.90, implying that there is some loss between the real and complex-number quantum encoding strategies.

For VoyageAI embeddings, the correlation coefficients are given in Table 2.

Table 2. VoyageAI embeddings correlation coefficients

	Classic	Q Real	Q Comp.int.	Q Comp.split
Classic		0.9756	0.9755	0.9731
Q Real			0.9535	0.9543
Q Comp.int.				0.9570

The correlation coefficients are obviously much higher in all cases. There is no significant difference between the classical and all versions of quantum embeddings.

We find little or no significant information loss using the quantum encoding approaches we suggest. It is important to note that any of the complex amplitude encoding strategies result in acceptable coefficients, in particular when using the Voyage embeddings.

Some loss in the mapping process can be located in all the operations performed. We expect a loss in the mapping from classical to quantum embeddings using amplitude encoding. [22] discuss the loss when using different encoding techniques, in particular amplitude encoding, using simulators and hardware quantum computers.

6.1 Availability

All code and data discussed in this article are available on the GitHub repository https://github.com/dcavar/nlqk as part of the Natural Language Qu Kit (NLQK) Python module. The models used are all freely available online. A link to the download sites is made available in the GitHub repo.

7 Conclusion

We show that the embeddings based on dense vectors with more than 1,000 dimensions and real numbers can be compressed to 12 qubits or less. The correlation coefficient was calculated using the classical similarity scores, and the SWAP-test scores of the quantum circuit representing the amplitude encoded embedding vectors show that there is a strong correlation between the classical and quantum similarities using these methods. This high correlation score

allows us to conclude that there is minimal information loss when mapping classical embeddings to quantum states. This means that classical word embedding models could potentially be used in quantum computing environments in NLP applications.

In some future quantum computing environments, the generation of new embedding models might still be necessary to exploit advanced vector space modeling potentials provided by the nature of qubits and potential embedding encodings. Current limitations and the lack of quantum memory or adequate ways for persistent storage of states prevent us from training purely quantum-based embedding models.

The possibility of using mappings of real numbers from embedding vectors to complex amplitudes in quantum states appears to be acceptable for semantic similarity computations in quantum computing. It provides an instrument to reduce the number of gates and the circuit depth even further. Using complex number encoding of embedding vectors reduces the linear scaling of input vectors to quantum circuits by 50%. Given the results from the similarity correlation coefficients, with some vector models, i.e., VoyageAI vectors, the loss is minimal when compared to the real-number amplitude encoding.

On the other hand, the interface between classical and quantum computing is still problematic. Data transfer of classical embedding vectors to amplitude-encoded quantum states is time-consuming and costly.

8 Limitations

We must emphasize that this work currently has various limitations. While the insight into the potential use of classical NLP and AI resources and models in quantum computers is highly relevant to our work, the limitations restrict the potential benefit in QNLP, given the current state of quantum computers.

Quantum computers still produce significant error rates that limit evaluations and experiments with NLP and AI algorithms in general on existing quantum hardware. Most work in the field is performed using quantum simulators, or it lacks a practical and empirical component. This error rate can affect amplitude encoding in particular. In addition to error rates, there are hardware limits on how precise the preparation of quantum states using amplitudes can be and how precise the reading of amplitudes can be. These are hardware and implementation restrictions that are beyond our control and beyond the scope of this work.

The amplitude encoding approach converts n-dimensional vectors of real numbers into quantum circuits with $log_2(n)$ or $log_2(n/2)$ qubits. However, the worst-case scenario is that the computed quantum circuit, to prepare the quantum state of these $log_2(n)$ qubits, will require a n number of quantum gates to set the state of the qubits. Given the mentioned error rate of current quantum computers, this is not desirable since every quantum gate might, and in fact, on current hardware will increase the error of the computation. A considerable depth of quantum circuits in terms of quantum gates might result in decoherence

effects and high error rates. It also consumes computation time in quantum computers. However, this problem can be partially remedied by using the complex number amplitude encoding strategy that reduces the number of gates and the circuit depth by 50%.

When mapping classical vectors to quantum states using only real-number amplitude encoding, we map vectors of real numbers to amplitudes that consequently stay real. That is, the imaginary part of complex amplitudes in qubits will be zero in this encoding strategy. If we were able to tune and train embeddings on a quantum computer, we would be able to use the full potential of complex number amplitude-encoded embeddings. To be able to apply any repetitive optimization cycle on quantum states, quantum RAM is needed, in addition to sufficient fault-tolerant qubits and persistence of states over longer time periods. Current quantum computer environments do not offer this.

Acknowledgement. This material is based upon work supported by the Center for Quantum Technologies under the Industry-University Cooperative Research Center Program at the US National Science Foundation under Grant No. 2224928.

This research was supported in part by Lilly Endowment, Inc., through its support for the Indiana University Pervasive Technology Institute.

References

1. Barenco, A., Berthiaume, A., Deutsch, D., Ekert, A., Jozsa, R., Macchiavello, C.: Stabilisation of quantum computations by symmetrisation (1996). https://arxiv.org/abs/quant-ph/9604028
2. Baroni, M., Zamparelli, R.: Nouns are vectors, adjectives are matrices: representing adjective-noun constructions in semantic space. In: Proceedings of the 2010 Conference on Empirical Methods in Natural Language Processing, EMNLP '10, pp. 1183–1193. Association for Computational Linguistics, USA (2010)
3. Bojanowski, P., Grave, E., Joulin, A., Mikolov, T.: Enriching word vectors with subword information. Trans. Assoc. Comput. Linguist. **5**, 135–146 (2017)
4. Brown, T.B., .: Language models are few-shot learners (2020). https://arxiv.org/abs/2005.14165
5. Buhrman, H., Cleve, R., Watrous, J., De Wolf, R.: Quantum fingerprinting. Phys. Rev. Lett. **87**(16), 167902 (2001). https://doi.org/10.1103/PhysRevLett.87.167902
6. Cavar, D., Parukola, K.R.: Word and text similarity using classical word embeddings in quantum NLP systems. In: 2025 IEEE International Conference on Acoustics, Speech, and Signal Processing Workshops (ICASSPW), pp. 1–5 (2025). https://doi.org/10.1109/ICASSPW65056.2025.11010999
7. Cavar, D., Zhang, C.: Semantic similarities using classical embeddings in quantum NLP. In: 2024 IEEE International Conference on Quantum Computing and Engineering (QCE), vol. 02, pp. 450–451 (2024). https://doi.org/10.1109/QCE60285.2024.10350
8. Coecke, B., de Felice, G., Meichanetzidis, K., Toumi, A.: Foundations for near-term quantum natural language processing (2020). https://doi.org/10.48550/arXiv.2012.03755

9. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. In: Burstein, J., et al. (eds.) Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pp. 4171–4186. Association for Computational Linguistics (2019)
10. Dirac, P.A.M.: The Principles of Quantum Mechanics. International Series of Monographs on Physics. Oxford University Press, Oxford, New York (1982)
11. Firth, J.R.: A synopsis of linguistic theory 1930–1955. In: Palmer, F.R. (ed.) Selected Papers of J.R. Firth 1952–1959. Longman, London (1968)
12. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press (2016). http://www.deeplearningbook.org
13. Harris, Z.: Distributional structure. Word **10**(23), 146–162 (1954)
14. Hill, F., Reichart, R., Korhonen, A.: SimLex-999: evaluating semantic models with (genuine) similarity estimation. Comput. Linguist. **41**(4), 665–695 (2015)
15. Joulin, A., Bojanowski, P., Mikolov, T., Jégou, H., Grave, E.: Loss in translation: learning bilingual word mapping with a retrieval criterion. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (2018)
16. Jurafsky, D., Martin, J.H.: Speech and Language Processing, 2nd edn. Prentice-Hall Inc., Upper Saddle River, NJ, USA (2009)
17. Lenci, A., Sahlgren, M.: Distributional Semantics. Studies in Natural Language Processing. Cambridge University Press (2023). https://doi.org/10.1017/9780511783692
18. Li, Q., Uprety, S., Wang, B., Song, D.: Quantum-inspired complex word embedding. In: Augenstein, I., et al. (eds.) Proceedings of the Third Workshop on Representation Learning for NLP, pp. 50–57. Association for Computational Linguistics, Melbourne, Australia (2018). https://doi.org/10.18653/v1/W18-3006. https://aclanthology.org/W18-3006
19. Maria Schuld, I.S., Petruccione, F.: An introduction to quantum machine learning. Contem. Phys. **56**(2), 172–185 (2015). https://doi.org/10.1080/00107514.2014.964942
20. Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space (2013). https://arxiv.org/abs/1301.3781
21. Mitsuda, N., et al.: Approximate complex amplitude encoding algorithm and its application to data classification problems. Phys. Rev. A **109**(5) (2024). https://doi.org/10.1103/physreva.109.052423
22. Munikote, N., Li, A., Liu, C., Stein, S.: Comparing quantum encoding techniques (2024). https://arxiv.org/abs/2410.09121
23. Pennington, J., Socher, R., Manning, C.: GloVe: global vectors for word representation. In: Moschitti, A., Pang, B., Daelemans, W. (eds.) Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), Doha, Qatar, pp. 1532–1543. Association for Computational Linguistics (2014). https://doi.org/10.3115/v1/D14-1162
24. Pennington, J., Socher, R., Manning, C.D.: GloVe: global vectors for word representation. In: Empirical Methods in Natural Language Processing (EMNLP), pp. 1532–1543 (2014). http://www.aclweb.org/anthology/D14-1162
25. Peral-García, D., Cruz-Benito, J., García-Peñalvo, F.J.: Comparing Natural Language Processing and Quantum Natural Processing approaches in text classification tasks. Exp. Syst. Appl. **254**, 124427 (2024). https://doi.org/10.1016/j.eswa.2024.124427

26. Rath, M., Date, H.: Quantum data encoding: a comparative analysis of classical-to-quantum mapping techniques and their impact on machine learning accuracy. arXiv arXiv:2311.10375 (2023). https://api.semanticscholar.org/CorpusID:265281517
27. Rumelhart, D.E., Abrahamson, A.A.: A model for analogical reasoning. Cogn. Psychol. **5**(1), 1–28 (1973). https://doi.org/10.1016/0010-0285(73)90023-6
28. Rumelhart, D.E., Todd, P.M.: Learning and connectionist representations. In: Meyer, D.E., Kornblum, S. (eds.) Attention and Performance 14: Synergies in Experimental Psychology, Artificial Intelligence, and Cognitive Neuroscience, pp. 3–30. The MIT Press (1993)
29. Schuster, M., Nakajima, K.: Japanese and Korean voice search. In: International Conference on Acoustics, Speech and Signal Processing, pp. 5149–5152 (2012)
30. Schütze, H.: Automatic word sense discrimination. Comput. Linguist. **24**(1), 97–124 (1998)
31. Speer, R., Chin, J., Havasi, C.: ConceptNet 5.5: an open multilingual graph of general knowledge. In: Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, pp. 4444–4451. AAAI'17. AAAI Press (2017)
32. Taylor, W.L.: "cloze procedure": a new tool for measuring readability. J. Mass Commun. Q. **30**, 415–433 (1953). https://api.semanticscholar.org/CorpusID:206666846
33. Vaswani, A., et al.: Attention is all you need (2023). https://arxiv.org/abs/1706.03762
34. Zhang, J., et al.: Applications of quantum embedding in computer vision. In: Luo, B., Cheng, L., Wu, Z.G., Li, Hongyi ad Li, C. (eds.) Neural Information Processing, pp. 165–177. Springer Nature Singapore, Singapore (2024)